

SKILL 19

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

pid_t foreground_pid = -1;
char job_status[20] = "None";

/* SIGINT handler for the shell */
void handle_sigint(int sig) {
    printf("\nShell caught SIGINT\n");

    if (foreground_pid > 0) {
        printf("Forwarding SIGINT to foreground process...\n");

        kill(foreground_pid, SIGINT);

        strcpy(job_status, "Terminated");
    } else {
        printf("No foreground job running\n");
    }
}
```

```
int main() {
    struct sigaction sa;

    printf("=== Task 19: Signal Handling ===\n");

    printf("\n=== Register Signal Handler ===\n");

    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("SIGINT handler registered successfully\n");

    printf("\n=== Create Foreground Job ===\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }
}
```

```

if (pid == 0) {
    /* Child restores normal SIGINT behavior */
    signal(SIGINT, SIG_DFL);

    printf("Foreground child started\n");
    printf("Child PID: %d\n", getpid());

    sleep(20);

    printf("Child completed normally\n");

    exit(0);
}

foreground_pid = pid;
strcpy(job_status, "Running");

printf("Foreground job PID: %d\n", foreground_pid);
printf("Job Status: %s\n", job_status);

printf("\n=== Interrupt Test ===\n");
printf("Press Ctrl+C now to test SIGINT handling\n");

int status;

waitpid(foreground_pid, &status, 0);

if (WIFSIGNALED(status)) {
    printf("\nForeground job terminated by signal %d\n",
        WTERMSIG(status));

    strcpy(job_status, "Terminated");
}
else if (WIFEXITED(status)) {
    printf("\nForeground job exited normally\n");

    strcpy(job_status, "Completed");
}

foreground_pid = -1;

printf("\n=== Updated Job State ===\n");
printf("Job Status: %s\n", job_status);

```

```

        strcpy(job_status, "Terminated");
    }
    else if (WIFEXITED(status)) {
        printf("\nForeground job exited normally\n");

        strcpy(job_status, "Completed");
    }

    foreground_pid = -1;

    printf("\n=== Updated Job State ===\n");
    printf("Job Status: %s\n", job_status);

    printf("\n=== Shell Protection Test ===\n");
    printf("Shell process is still running\n");
    printf("SIGINT affected the foreground job only\n");

    printf("\n=== Signal Test Summary ===\n");
    printf("Signal handler registered\n");
    printf("SIGINT captured\n");
    printf("Signal forwarded to foreground job\n");
    printf("Shell process protected\n");
    printf("Foreground job terminated\n");
    printf("Job state updated\n");

    printf("\n=== Task 19 Complete ===\n");

    return 0;
}

```

Output:

```

=== Task 19: Signal Handling ===

=== Register Signal Handler ===
SIGINT handler registered successfully

=== Create Foreground Job ===
Foreground job PID: 524
Job Status: Running

```

```
=== Interrupt Test ===  
Press Ctrl+C now to test SIGINT handling  
Foreground child started  
Child PID: 524  
^C  
Shell caught SIGINT  
Forwarding SIGINT to foreground process...  
  
Foreground job terminated by signal 64
```

```
=== Updated Job State ===  
Job Status: Terminated  
  
=== Shell Protection Test ===  
Shell process is still running  
SIGINT affected the foreground job only  
  
=== Signal Test Summary ===  
Signal handler registered  
SIGINT captured  
Signal forwarded to foreground job  
Shell process protected  
Foreground job terminated  
Job state updated
```